



WELCOME





C++ Loops and Arrays

Agenda

C++ Bridge Course

Loops

Arrays

Practice

Loops

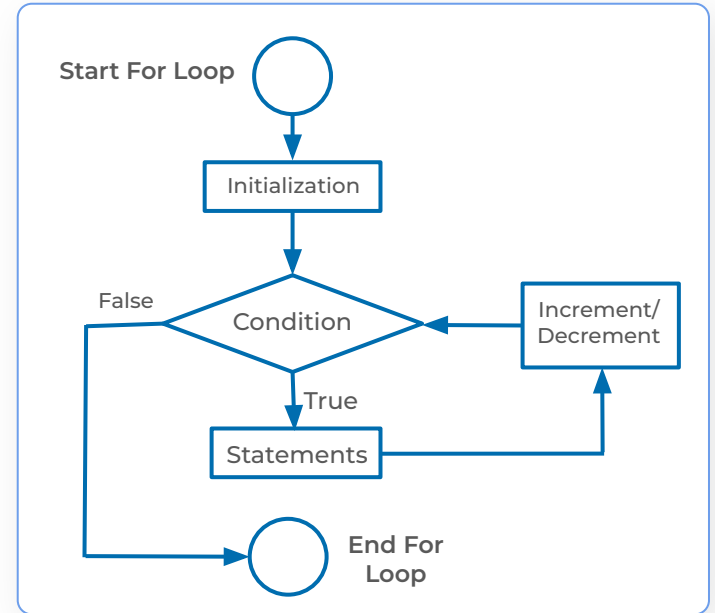


v/s



For Loop

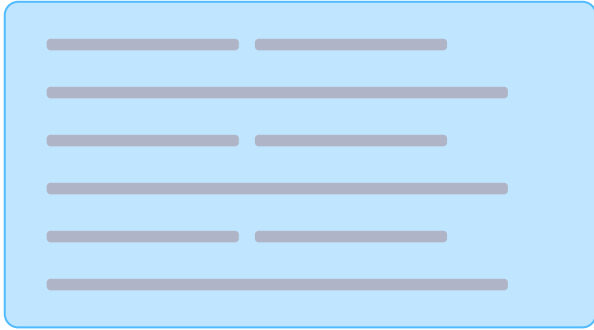
A **For loop** in programming is used to **repeatedly execute** a **block** of **code** a **specific** number of **times**, controlled by an initial condition, a condition for continuation, and an update statement.



Syntax: C++ Vs Python

For Loop

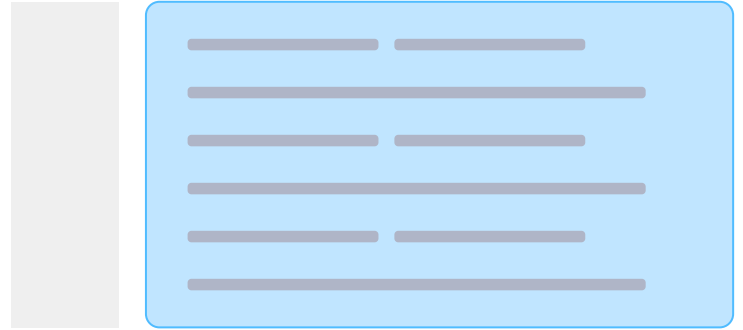
for (initialization; condition; update) {



}



for variable in range(start, stop, step):



For Loop

```
for( int i = 1 ; i<5 ; i = i+1 )  
{ cout << i << endl; }
```

Iteration	i Value Before Update	Condition (i < 5)	Action (Print i)	Updated i Value
1	1	TRUE	Print 1	2
2	2	TRUE	Print 2	3
3	3	TRUE	Print 3	4
4	4	TRUE	Print 4	5
5	5	FALSE		

For Loop

C++



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    for(int i=1; i<5; i++){
        cout << i << endl;
    }
    return 0;
}
```

Output



1

2

3

4

Syntax: C++ Vs Python

For Loop (Range-Based)

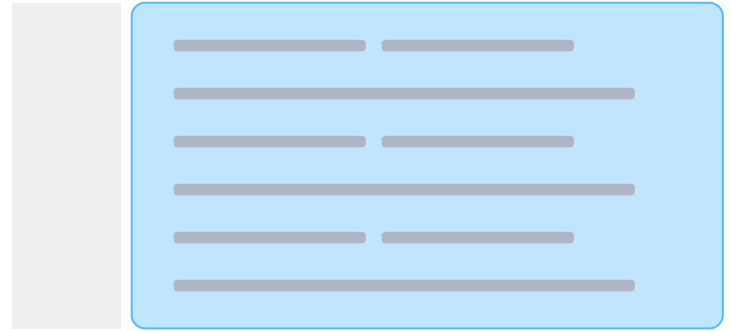
```
for (type each_item : sequence) {
```



```
}
```



```
for each_item in sequence:
```



Syntax: C++ Vs Python

For Loop (Range-Based)

C++ 

```
int num_seq[] = {1, 2, 3, 4, 5};  
for (int each_item : num_seq) {  
    cout << each_item << endl;  
}
```

Python 

```
num_seq = [1, 2, 3, 4, 5]  
for each_item in num_seq:  
    print(each_item)
```

For Loop (Range-Based)

Example

C++ 

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int num_seq[] = {1, 2, 3, 4, 5};
    for (int each_item : num_seq) {
        cout << each_item << endl;
    }
    return 0;
}
```

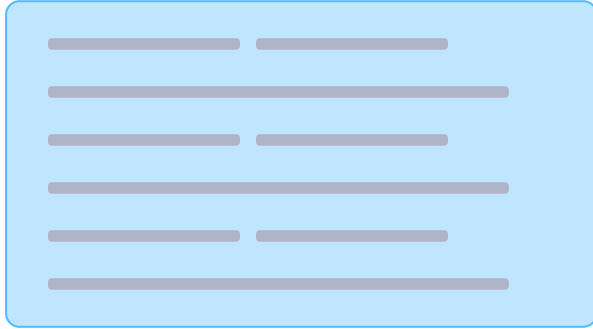
Output 

1
2
3
4
5

Syntax: C++ Vs Python

While Loop

```
while (condition) {
```



```
}
```



```
while condition:
```



While Loop

C++ 

```
int i = 1;
while (i <= 5) {
    cout << i << endl;
    i = i + 1;
}
```

Python 

```
i = 1
while i <= 5:
    print(i)
    i = i + 1
```

While Loop

Example

C++ 

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int i = 1;
    while (i ≤ 5) {
        cout << i << endl;
        i = i + 1;
    }
    return 0;
}
```

Output 

1
2
3
4
5

Do-While Loop

C++



```
int i = 1;
do {
    cout << i << endl;
    i = i + 1;
} while (i <= 5);
```

Do-While Loop

C++



```
int i = 1;
```

```
do {
```

```
    cout << i << endl;
```

```
    i = i + 1;
```

```
} while (i ≤ 5);
```

***Initializes the variable *i* to 1
before the loop starts.***

Do-While Loop

C++



```
int i = 1;  
do {  
    cout << i << endl;  
    i = i + 1;  
} while (i ≤ 5);
```

*Marks the beginning of the do-while loop, which will **execute at least once** before checking the condition.*

Do-While Loop

C++



```
int i = 1;
do {
    cout << i << endl;
    i = i + 1;
} while (i ≤ 5);
```

Prints the current value of i on the screen.

Do-While Loop

C++



```
int i = 1;
do {
    cout << i << endl;
    i = i + 1;
} while (i ≤ 5);
```

***Increments** the value of **i** by **1** to update it for the next iteration.*

Do-While Loop

C++



```
int i = 1;
do {
    cout << i << endl;
    i = i + 1;
} while (i ≤ 5);
```

*After **each iteration**, checks if **i** is **less than or equal to 5**. If true, the loop continues. If false, the loop ends.*

Example

C++ 

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int i = 1;
    do {
        cout << i << endl;
        i = i + 1;
    } while (i <= 5);
    return 0;
}
```

Output 

1
2
3
4
5

Loop Control Statements



v/s



C++

Loop Control Statements

- ✓ Control statements **alter** the sequential **execution** of a program.
- ✓ In both Python and C++, loop control statements **break** and **continue** work similarly.



Break Statement

Example

C++ 

```
#include <iostream>
using namespace std;
int main() {
    int num_seq[] = {1, 2, 3, 4, 5};
    for (int i = 0; i < 5; i++) {
        if (num_seq[i] == 4) {
            break;
        }
        cout << num_seq[i] << endl;
    }
    return 0;
}
```

Output 

1
2
3

The break statement is used to **stop a loop and move on to the next** part of the program.

Example

C++ 

```
#include <iostream>
using namespace std;
int main() {
    int num_seq[] = {1, 2, 3, 4, 5};
    for (int i = 0; i < 5; i++) {
        if (num_seq[i] % 2 == 1) {
            continue;
        }
        cout << num_seq[i] << endl;
    }
    return 0;
}
```

Output 

2

4

The continue statement is **used to skip the current iteration** of a loop and move to the next one.

Quiz Time!



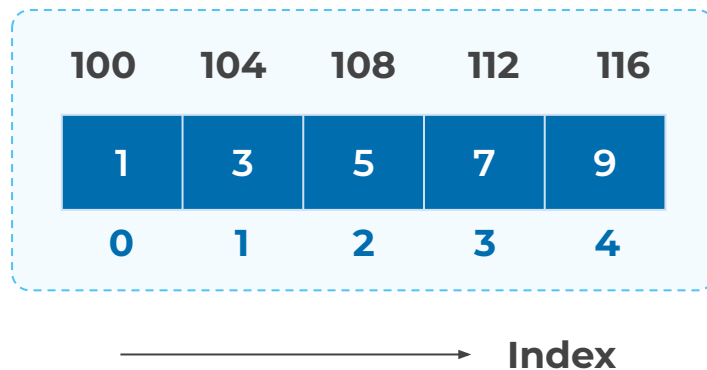
Arrays



Arrays

- ✓ An **array in C++** is a collection of **elements** of the same type, stored in contiguous memory locations. It allows easy **storage and manipulation** of multiple values of the same type.

Contiguous *memory location*.



Declaration of Arrays

C++ Array Declaration

In C++, the size of the array must be specified at the time of declaration (except for dynamically allocated arrays).

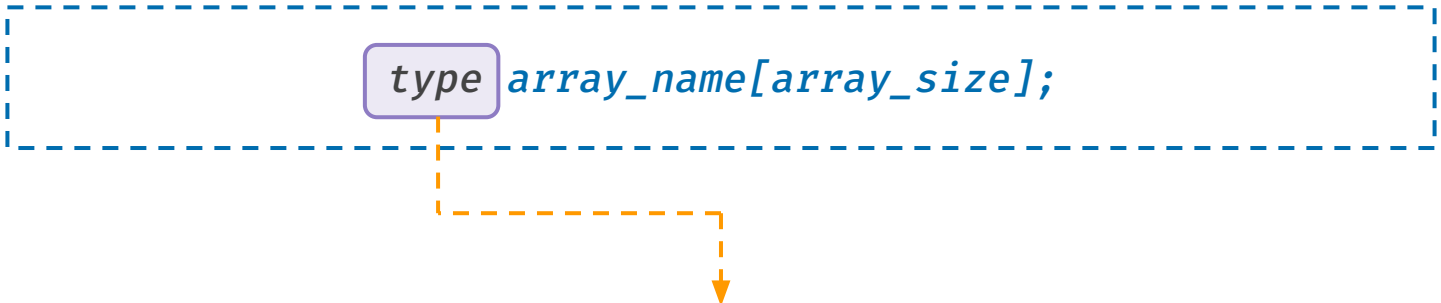
```
int num_seq[5]; // Declaring an array of 5 integers
```

Python Array Declaration

In Python, arrays are usually implemented using lists, and size is not fixed.

```
num_seq = [1, 2, 3, 4, 5] # List in Python
```

Syntax of Array Declaration in C++



```
type array_name[array_size];
```

Specifies the data type
(e.g., **int**, **float**, **char**).

Syntax of Array Declaration in C++

```
type array_name [array_size];
```



Name of the array.

Syntax of Array Declaration in C++

```
type array_name [array_size];
```

The **number of elements** the
array can hold.

Syntax in C++

```
type array_name[array_size];
```

Example in C++

```
int num_seq[5] = {1, 2, 3, 4, 5}; // Array of integers with 5 elements
```

C++

How to Take Array Input



To take user input in C++ for an array, we use a for loop along with the cin function to store the values.

How to Take Array Input

C++ 

```
#include <iostream>
using namespace std;
int main() {
    int num_seq[5]; // Declare array of size 5
    for (int i = 0; i < 5; i++) {
        cout << "Enter element " << i + 1 << ": ";
        cin >> num_seq[i];
    }
    for (int i = 0; i < 5; i++) {
        cout << num_seq[i] << " ";
    }
    return 0;
}
```

To take user input in C++ for an array, we use a for loop along with the cin function to store the values.

Quiz Time!



Key Takeaways

✓ ***Loops in C++***

Explanation of for, while, and do-while loops, along with range-based for loops for better readability.

✓ ***Loop Control Statements***

Covers break and continue, demonstrating their effect on loop execution.

✓ ***Working with Arrays***

Covers array declaration, input handling, and iterating over arrays using both for-loops and range-based loops.





THANK YOU





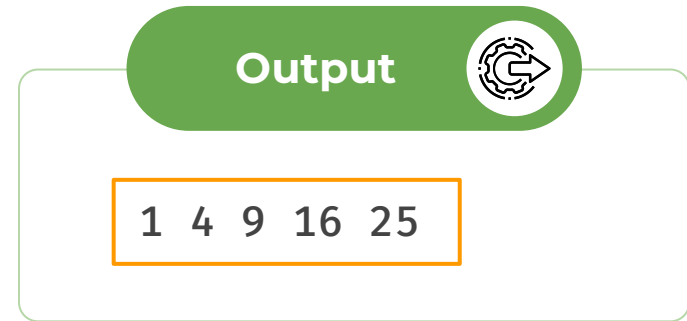
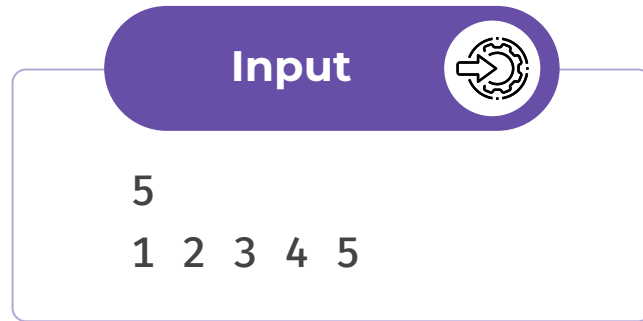
ALL THE BEST



Reading array as Input

Problem Statement

★ *Write a program that reads user input and prints the square of each integer.*



Here, we are required to print the square of each integer with the
results separated by a space.

C++

Problem Solution

C++



```
#include<bits/stdc++.h>
using namespace std;
int main() {
    int size;
    cin >> size;
    int num_seq[size];
    for(int i = 0; i < size; i++) {
        cin >> num_seq[i];
    }
    for (int each_item: num_seq) {
        cout << each_item * each_item << " ";
    }
    return 0;
}
```

Input



5

1 2 3 4 5

Output



1 4 9 16 25

Practice Problem

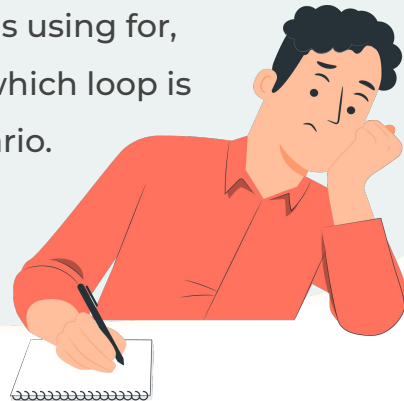


v/s



Repeated Inputs

Write a program that **repeatedly prompts the user** for a **positive number** and **prints** it. If the input is zero or negative, the program should continue asking until a valid number is entered. Implement this using for, while, and do-while loops, and discuss which loop is the most intuitive for this scenario.



Repeated Inputs

Using for Loop

C++



```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int num;
    for (;;) {
        cout << "Enter a positive number" << endl;
        cin >> num;
        if (num > 0) {
            cout << num << endl;
            break;
        }
    }
    return 0;
}
```

Repeated Inputs

Using while Loop

C++



```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int num;
    while (1) {
        cout << "Enter a positive number" << endl;
        cin >> num;
        if (num > 0) {
            cout << num << endl;
            break;
        }
    }
    return 0;
}
```

Repeated Inputs

Using do-while Loop

C++



```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int num;
    do {
        cout << "Enter a positive number" << endl;
        cin >> num;
    } while (num <= 0);

    cout << num << endl;
    return 0;
}
```